

Linux eBPF Process Lifecycle Monitor

گزارش دوزبانه پروژه / Bilingual project report

Repository: <https://github.com/Hesam-deb/linux-ebpf-exec-monitor>

Report date: June 22, 2026

Stack: Linux eBPF, BCC, Python, Flask, HTML/CSS

1. Project objective

The project started as a small execution monitor that reported PID, command name, and timestamp. It has been expanded into a process lifecycle monitor with a bilingual Persian/English dashboard and a documented installation workflow.

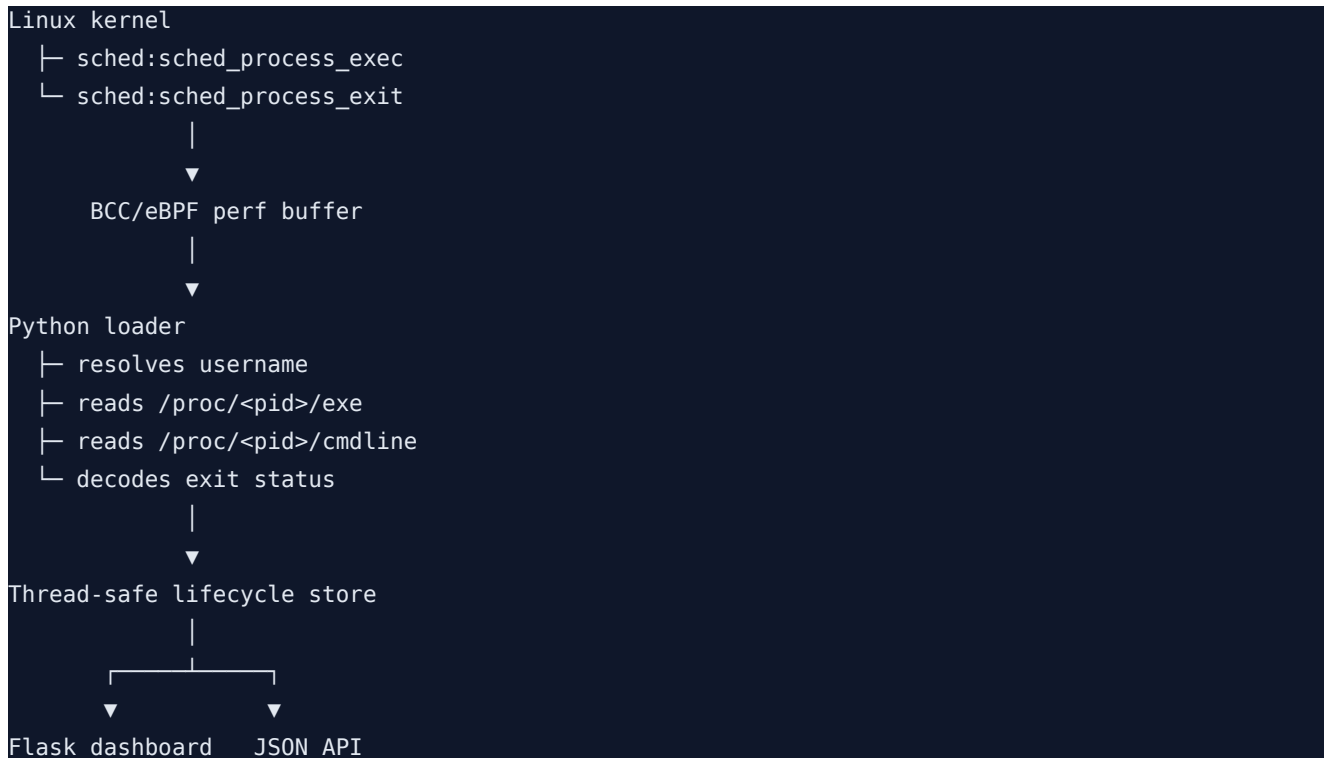
2. What was implemented

Area	Completed work
Kernel monitoring	Capture successful process execution and process exit with <code>sched_process_exec</code> and <code>sched_process_exit</code> .
Identity	Capture PID, parent PID, UID, and resolve the local user name.
Invocation	Read executable path and command-line arguments from <code>/proc/<pid></code> on a best-effort basis.
Lifecycle	Correlate start and exit events, calculate duration, and decode exit code or terminating signal.
Dashboard	Responsive dark UI, Persian and English languages, RTL/LTR layouts, live one-second polling, persistent expanded details, eBPF pipeline-load metrics, and JSON output.
Documentation	Friendly English and Persian READMEs with HTTPS/SSH clone addresses,

requirements, installation, troubleshooting, and limitations.

Quality Automated model and Flask tests for lifecycle correlation, serialization, translations, and API behavior.

3. Architecture



4. Process details now available

- PID and parent PID (PPID)
- UID and user name
- Command name, executable path, and arguments
- Running or exited status
- Start time, exit time, and duration
- Exit code or termination signal

Live eBPF pipeline load

The dashboard reports a rolling event rate, exec/exit totals, lost perf-buffer events, and buffer health. These values measure event-pipeline pressure rather than claiming to

measure exact kernel instruction CPU cost. Data updates without a page reload, so expanded process details remain open.

“What the process is doing” is intentionally scoped to lifecycle information. Monitoring file

access, network traffic, or every system call would require additional probes, filtering, and

higher-volume storage.

5. Running the project

```
git clone https://github.com/Hesam-deb/linux-ebpf-exec-monitor.git
cd linux-ebpf-exec-monitor
chmod +x scripts/install.sh scripts/run.sh
./scripts/install.sh
sudo ./scripts/run.sh
```

Persian dashboard: <http://127.0.0.1:5000>

English dashboard: <http://127.0.0.1:5000/?lang=en>

JSON API: <http://127.0.0.1:5000/api/events>

6. Requirements and constraints

- Ubuntu 24.04 or a recent Ubuntu/Debian-based Linux distribution
- Linux kernel 6.x or newer, matching kernel headers, BCC packages, Python 3, Flask 3.x, and root privileges
- Windows and macOS require an Ubuntu virtual machine
- Events are retained in memory and are lost when the application stops
- `/proc` enrichment can miss very short-lived processes
- The dashboard is educational and has no authentication or TLS

گزارش پروژه پایش چرخه عمر پردازشها

با eBPF

مخزن: <https://github.com/Hesam-deb/linux-ebpf-exec-monitor>

تاریخ گزارش: ۲۲ ژوئن ۲۰۲۶

فناوریها: Linux eBPF, BCC, Python, Flask و HTML/CSS

هدف پروژه ۱.

نام دستور و زمان اجرا بود. اکنون به پایشگر چرخه عمر پردازش PID، پروژه ابتدا یک پایشگر ساده برای نمایش دارد JSON خروجی API تبدیل شده و داشبورد دوزبانه فارسی/انگلیسی، راهنمای نصب و

قابلیت‌های اضافه‌شده ۲.

- هسته لینوکس tracepoint ثبت اجرای موفق و خروج پردازش با دو
- به نام کاربر UID و تبدیل PID، PPID و UID ثبت
- خواندن مسیر فایل اجرایی و آرگومان‌ها از `/proc`
- تطبیق رویداد شروع و پایان و محاسبه مدت اجرا
- نمایش کد خروج یا سیگنال پایان
- و به‌روزرسانی زنده یک‌ثانیه‌ای RTL/LTR داشبورد واکنش‌گرا با زبان فارسی و انگلیسی، حالت
- JSON جزئیات بازشونده برای هر پردازش و خروجی
- perf buffer حفظ پنل جزئیات بازشده و نمایش نرخ رویداد، رویدادهای از دست‌رفته و سلامت
- و رابط دوزبانه API، آزمون‌های خودکار برای مدل

مسیر داده ۳.

اطلاعات Python ارسال می‌شوند. بارگذار Python به perf buffer رویدادهای اجرا و خروج در هسته ثبت و از تطبیق می‌دهد و PID رویداد شروع و پایان را با thread-safe را اضافه می‌کند. سپس حافظه `/proc` کاربر و نمایش می‌دهد API نتیجه را در داشبورد و Flask.

۴. اطلاعات قابل مشاهده برای هر پردازش

- والد PID و PID
- UID نام کاربر و
- نام دستور، مسیر فایل اجرایی و آرگومان‌ها
- وضعیت در حال اجرا یا پایان یافته
- زمان شروع، زمان پایان و مدت اجرا
- کد خروج یا سیگنال پایان

eBPF بار زنده خط لوله

را perf buffer داشبورد نرخ متحرک رویدادها، مجموع رویدادهای اجرا و خروج، رویدادهای از دست رفته و سلامت در eBPF دستورهای CPU نشان می‌دهد. این معیارها فشار خط لوله رویداد را اندازه می‌گیرند، نه مصرف دقیق. هسته، به روزرسانی بدون بارگذاری دوباره صفحه انجام می‌شود و جزئیات باز می‌مانند.

ها به	system call طور از فعالیت پردازش در این نسخه، چرخه عمر آن است. بررسی فایل‌ها، شبکه و همه
-------	--

probe	ه.ی جداگانه و زیرساخت ذخیره‌سازی قوی‌تری نیاز دارد
-------	--

۵. اجرای پروژه

```
git clone https://github.com/Hesam-deb/linux-ebpf-exec-monitor.git
cd linux-ebpf-exec-monitor
chmod +x scripts/install.sh scripts/run.sh
./scripts/install.sh
sudo ./scripts/run.sh
```

داشبورد فارسی: <http://127.0.0.1:5000>

داشبورد انگلیسی: <http://127.0.0.1:5000/?lang=en>

API: <http://127.0.0.1:5000/api/events>

۶. محدودیت‌ها

- و ذخیره‌سازی دائمی ندارد TLS، پروژه آموزشی است و احراز هویت

- برای پردازش‌های بسیار کوتاه ممکن است از دست برود `/proc` اطلاعات
- نیاز است root هدرهای هسته و دسترسی، BCC، Linux، برای اجرا به
- پایش فایل و شبکه در این نسخه انجام نمی‌شود

Linux eBPF Process Lifecycle Monitor — Bilingual Project Report